Procurement Analytics

This is a case study proposed for analyzing a sample data from procurement process. The company wants to analyze if it is good option to use 'Contract' or 'Spot' buying mode.

Analyzing the file `purchase_metadata_SAMPLE.xlsx`

Let's see the characteristics of the columns of the data. With `df.info()` we could see if there is empty values and the data type for each column. This is so important in order to clean, transform and complete the data.

```
p_m.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 165 entries, 0 to 164
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   POSampleID        165 non-null    str
 1   SupplierCode      165 non-null    str
 2   MaterialCode      165 non-null    str
 3   Center            165 non-null    str
 4   OrderDate         165 non-null    str
 5   BuyingMode        165 non-null    str
 6   OrderQuantity     165 non-null    int64
 7   UnitPriceUSD      165 non-null    float64
 8   TransportCostUSD  161 non-null    float64
 9   DaysInStorage     160 non-null    float64
 10  BatchNumber       165 non-null    str
 11  NegotiatorID      153 non-null    str
dtypes: float64(3), int64(1), str(8)
memory usage: 15.6 KB
```

Structure of the `MaterialCode` column

```
# Analyzing the structure of the 'MaterialCode' column
p_m['MaterialCode'].value_counts()
```

```
MaterialCode
MAT-01015    12
MAT-1014     10
MAT-1001      9
MAT-1008      9
MAT-1005      9
MAT-1023      9
MAT-1021      9
MAT-1007      8
MAT-1010      8
MAT-1006      7
MAT-1018      7
MAT-1019      6
```

```
MAT-1016      6
MAT-1011      6
MAT-1017      6
MAT-1003      6
MAT-1002      6
MAT-1024      6
MAT-1012      5
MAT-01004     4
MAT-1009      4
MAT-1022      4
MAT-1013      3
MAT-1020      3
MAT-1025      3
Name: count, dtype: int64
```

Since `MaterialCode` has some inconsistencies, first it is necessary to normalize the codes for `MAT-01004` to `MAT-1004` and `MAT-01015` to `MAT-1015`, in order to have the same structure for all, and to have consistency with the list of `MaterialCode` in `material_class.csv` file.

```
# 'MaterialCode' values in 'purchase_metadate.xlsx'
sorted(p_m['MaterialCode'].unique())
```

```
['MAT-1001',
 'MAT-1002',
 'MAT-1003',
 'MAT-1004',
 'MAT-1005',
 'MAT-1006',
 'MAT-1007',
 'MAT-1008',
 'MAT-1009',
 'MAT-1010',
 'MAT-1011',
 'MAT-1012',
 'MAT-1013',
 'MAT-1014',
 'MAT-1015',
 'MAT-1016',
 'MAT-1017',
 'MAT-1018',
 'MAT-1019',
 'MAT-1020',
 'MAT-1021',
 'MAT-1022',
 'MAT-1023',
 'MAT-1024',
 'MAT-1025']
```

Now, all values of `MaterialCode` column in `purchase_metadata_SAMPLE.xlsx` file are normalized, and coincide with the values in `MaterialCode` column in `material_classification_SAMPLE.csv` file.

Now, it's required to filter the registers in the required period of time between January and September 2024.

```
# Filtering the data by 'OrderDate', between January and September 2024
p_m = p_m[p_m['OrderDate'].between('2024-01-01', '2024-09-30')]
```

Check consistency between `POSampleID` suffix with `Center` values

I will assume the suffix of `POSampleID` as the correct data for `Center` values. So let's transform the column based on this assumption.

```
# Assign the suffix of 'POSampleID' as value for 'Center'
p_m['Center'] = p_m['POSampleID'].str.split('-').str[1]
```

Missing Values on `TransportCostUSD`, `DaysInStorage` & `NegotiatorID`

There are three columns with missing values: `TransportCostUSD`, `DaysInStorage` & `NegotiatorID`.

For `NegotiatorID` it will assumed that the information is not critical for the analysis required in this case study. So the rows with this missing values in `NegotiatorID` will stand in the data.

For `TransportCostUSD` and `DaysInStorage` the rows with missing values will be dropped. It is because the total number of rows for this case is **9**, which represents only **5,5 %** of total rows. Furthemore, of this total **5 rows** corrsponds to *Contract* `BuyingMode` and **4 rows** correspond to *Spot* `BuyingMode`. This is a balanced droping, so it will not affect considerably the analysis required.

Based on the previous assumptions, the `purchase_metadata_SAMPLE.xlsx` dataset now has 155 rows.

```
p_m.dropna(subset=['TransportCostUSD', 'DaysInStorage'], inplace=True)
p_m.shape
```

```
(155, 12)
```

After this, the `purchase_metadata_SAMPLE.xlsx` dataset is normalized, filtered and cleaned. It is ready to use for the required analysis.

Calculation of the per unit cost of every Purchase Order

For this calculation lets define the following variables (those highlitghed in gray are given in the data set):

- `PerUnitTransportCost`: It is the transportation cost per unit.
  - `PerUnitTransportCost` = `TransportCostUSD` / `OrderQuantity`
- `PerUnitStorageCost`: It is the storage cost per unit.
  - `PerUnitStorageCost` = `DaysInStorage` * `StorageCostPerUnitPerDay`
- `POUnitCost`: It is the total cost per unit for each purchase order.
  - `POUnitCost` = `UnitPriceUSD` + `PerUnitTransportCost` + `PerUnitStorageCost`

`PerUnitStorageCost`

Given the columns `DaysInStorage` and `StorageCostPerUnitPerDay` it will be calculated a new column `PerUnitStorageCost` in order to define how much it cost the storage of each unit of the respective purchase order before it was consumed.

`POUnitCost`

Now it will be calculated the `POUnitCost`, which is the per unit cost of each purchase order. This is relevant because it allows to compare the cost for each material based on the two different buying modes (contract vs. spot). This cost is the sum of `UnitPriceUSD` , `PerUnitTransportCost` and `PerUnitStorageCost`.

`DaysLeft` for Identifying Expired Products

It will be created a column for calculating the `DaysLeft` until expiration of the product. Those products in which the `DaysInStorage` have surpased the `ShelfLifeDays` the result will be negative, so it could be assumed the product is expired.

Business & Procurement Interpretation

The question proposed for this analysis is: - What can this data tell us about whether opportunistic buying is cost-effective, and under what conditions?

For answering this question the dataset will be filtered and sorted based on the following conditions: - Remove those rows whitout `POUnitCost`. - Show only columns `MaterialCode`, `BuyingMode`, `DaysLeft`, `UnitPriceUSD`, `POUnitCost`, `POsampleID`. - Order the filtered dataset based on 1.`MaterialCode`, 2.`BuyingMode`, 3.`POUnitCost`.

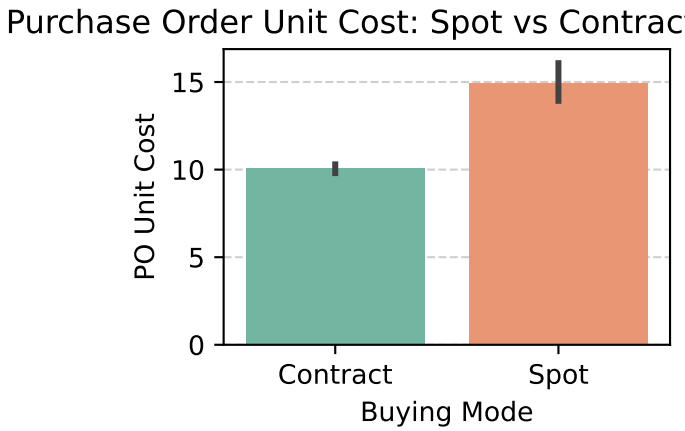
	POSampleID	MaterialCode	BuyingMode	DaysLeft	UnitPriceUSD	POUnitCost
32	PO10033-S	MAT-1001	Contract	56.0	31.72	33.803685
2	PO10003-P	MAT-1001	Contract	61.0	31.99	34.096043
27	PO10028-S	MAT-1001	Contract	53.0	32.11	34.471964
7	PO10008-P	MAT-1001	Contract	54.0	33.20	35.294346
96	PO10097-S	MAT-1001	Contract	39.0	33.13	36.632100
...	...	...	...	...	...	...
102	PO10103-N	MAT-1018	Contract	2349.0	33.01	34.050619
48	PO10049-S	MAT-1018	Contract	2317.0	32.44	34.062792
126	PO10127-P	MAT-1018	Contract	2347.0	33.33	34.629276
147	PO10148-P	MAT-1018	Contract	2350.0	33.61	35.019924
24	PO10025-N	MAT-1018	Spot	2245.0	25.91	28.000225

Cases when Opportunistic Buying is NOT Cost-Effective

1. Purchase Orders for **MAT-1005**:

- In the **Spot** BuyingMode the **minimum** value for POUnitCost is **13.47**, while the **maximum** value for **Contract** BuyingMode is **10.38**. It is a difference of **3.09** in the POUnitCost.
- Event though the UnitPriceUSD for Spot BuyingMode is smaller than Contract BuyingMode, the fact that the product is expired generates the final POUnitCost be greater in the Spot case.
- This is a material from Raw Materials category.
- In this case the opportunistic buying is not cost-effective.

```
sns.barplot(
    data=(p_m_w[(p_m_w['POUnitCost'].notnull()) & (p_m_w['MaterialCode'] == 'MAT-1005')]
        [['POSampleID', 'MaterialCode', 'BuyingMode', 'DaysLeft', 'UnitPriceUSD', 'POUnitCost']]
        .sort_values(by=['MaterialCode', 'BuyingMode', 'POUnitCost'])),
    x='BuyingMode',
    y='POUnitCost',
    palette='Set2'
)
plt.title('Purchase Order Unit Cost: Spot vs Contract [MAT-1005]')
plt.xlabel('Buying Mode')
plt.ylabel('PO Unit Cost')
plt.xticks()
plt.grid(True, axis='y', linestyle='--', alpha=0.6)
plt.gca().set_axisbelow(True)
plt.show()
```



2. Purchase Orders for **MAT-1003**:
   - In the **Spot** BuyingMode the **minimum** value for POUnitCost is **12.35**, while the **maximum** value for **Contract** BuyingMode is **7.00**. It is a difference of **5.35** in the POUnitCost.
   - This is also a material from Raw Materials category.
   - In this case the opportunistic buying is not cost-effective.
3. **Conclusion:** When the value ShelfLifeDays is relatively small (less than 50 days), it is mainly for Raw Materials, the opportunistic buying at the end **would not be cost-effective**.

Cases when Opportunistic Buying is Cost-Effective

1. Purchase Orders for **MAT-1010**:
   - In this case the maximum Spot BuyingMode POUnitPrice is smaller than the minimum Contract BuyingMode cost, it is a **difference** of at least **1.28** (at most 6.87).
2. Purchase Orders for **MAT-1014**:
   - In this case de **difference** between Spot and Contract is at least **3.84** (at most 6.15).
3. **Conclusion:** For materials that are not in the Raw Materials category, it is mainly those which doesn't have short `ShelfLifeDays` (greater than 1000 days), the opportunistic buying **is cost-effective**.

Analysis of the Center Storage Capacity

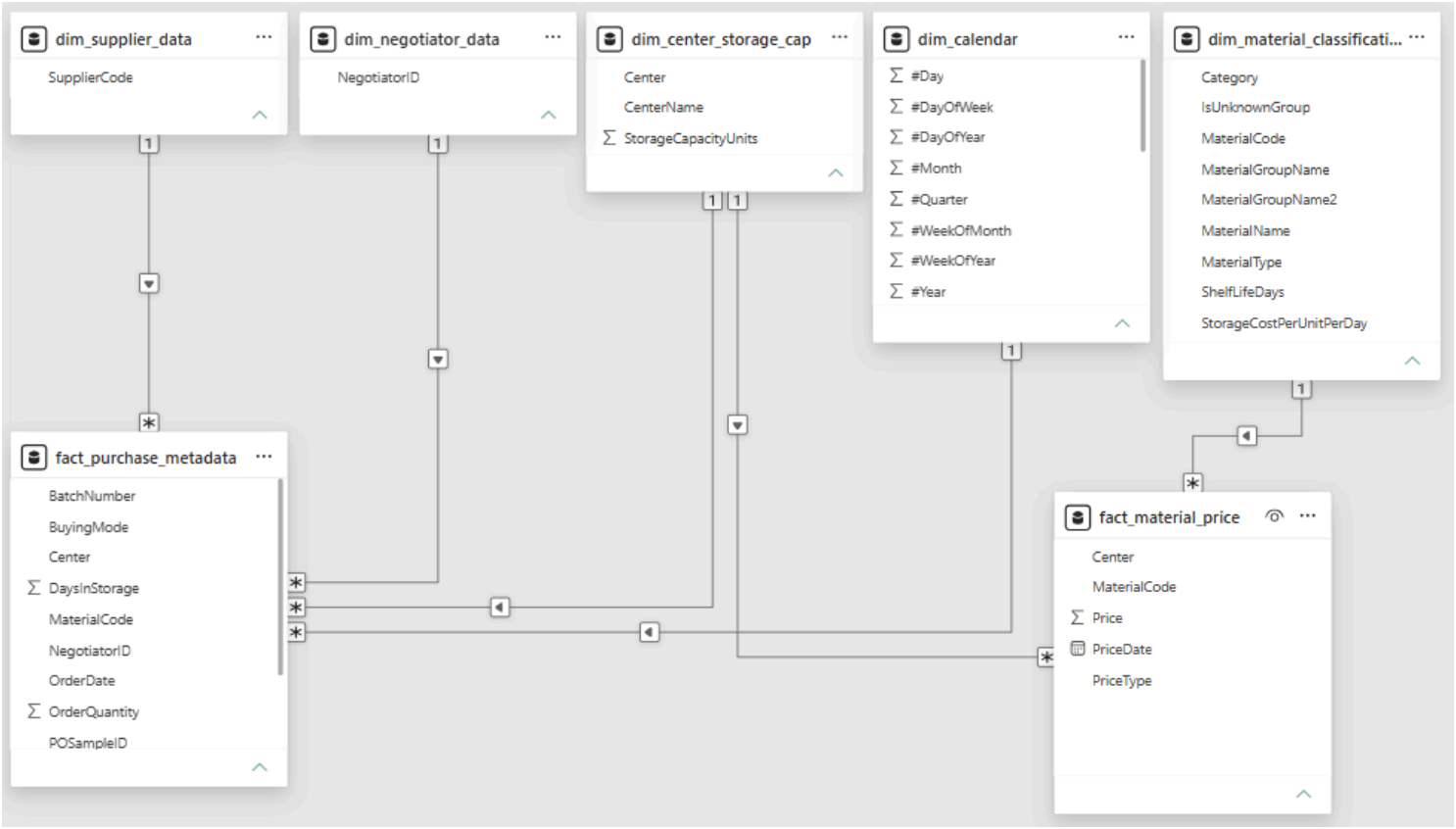
There are three outliers in the Purchase Order dataset. The following table shows: - PO10072-P, with OrderQuantity=19043, which is greater of the StorageCapacity of Center P: 18000 - PO10139-S, with OrderQuantity=43496, which is greater of the StorageCapacity of Center P: 42000 - PO10156-N, with OrderQuantity=60785, which is greater of the StorageCapacity of Center P: 60000

	POSampleID	MaterialCode	BuyingMode	Center	OrderQuantity
71	PO10072-P	MAT-1013	Spot	P	19043
138	PO10139-S	MAT-1005	Spot	S	43496
155	PO10156-N	MAT-1007	Spot	N	60785

As the Purchase Orders are in different dates, it is assumed this doesn't interfere in the previous analysis, because the capacity of Centers could be shared in order to take advantage of the Spot buying mode and the lower price.

Data Modeling and Structure

In the following image it is proposed the data model for this case study.



The model is a star schema type, and it is composed by:

1. **Fact Tables**
   - fact_purchase_metadata:
     - POSampleID
     - SupplierCode
     - MaterialCode
     - Center
     - OrderDate
     - BuyingMode
     - OrderQuantity
     - UnitPriceUSD
     - TransportCostUSD
     - DaysInStorage
     - BatchNumber
     - NegotiatorID
   - fact_material_price:
     - MaterialCode
     - Center
     - PriceType
     - Price
     - PriceDate
2. **Dimensions**
   - dim_material_classification:
     - MaterialCode, MaterialName, Category, MaterialGroupName, MaterialGroupName2, MaterialType, IsUnknown-Group, ShelfLifeDays, StorageCostPerUnitPerDay
   - dim_center_storage_cap:
     - Center, CenterName, StorageCapacityUnits
   - dim_supplier_data

    - SupplierCode
- dim_negotiator_data
    - NegotiatiorID
- dim_calendar
    - Date, #Year, #Month, #Day, …